

Four Authorship Roles — Schema, Rule, Cell, Initial Content: A Standalone Treatment of the Authorship Action Categories in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the four authorship roles that CKS deployments require — schema authoring, rule authoring, cell authoring, and initial content authoring — as a standalone architectural specification, so that downstream work distinguishing authorship from governance can do so without conflating the four roles with each other or with adjacent role categorizations.

Abstract

The CKS pattern's accessibility commitment per Claim 5 (§7.4 of the source paper) distinguishes non-specialist *governance* from non-specialist *authorship*. The governance side has been formalized as an architectural property in the parent foundational note on non-specialist governance (Li, 28 April 2026); the authorship side has been enumerated in that note's section 3 as a list of role categories inside the parent note's positioning argument. This note formalizes the authorship side as a standalone architectural specification. It identifies four authorship roles — schema authoring (producing substrate structure), rule authoring (producing orchestration rules), cell authoring (producing cell behavior), and initial content authoring (producing initial substrate content) — that correspond to the four categories of architectural elements every CKS deployment requires. It states each role precisely, names four operational components of the specification (role-specific architectural element, role-specific expertise requirement, role-specific architectural-commitment-location, role-specific relationship to governance), distinguishes the four roles from four adjacent role categorizations commonly conflated with them, names eight failure modes that collapse or conflate the roles, and provides an operational test for whether a deployment's authorship-role architecture satisfies the specification.

1. Why the four authorship roles need standalone formalization

The parent foundational note (Li, 28 April 2026) commits to non-specialist governance — the architectural property that the rights to inspect, modify, and override substrate content and orchestration rules are exercisable by non-specialists in commodity tools — and decouples that commitment from non-specialist authorship, which the architecture neither requires nor forbids. To make the asymmetry architectural rather than rhetorical,

that note's section 3 enumerated the kinds of authorship work CKS systems contain: substrate schema authorship, orchestration rule authorship, cell construction, and initial substrate content authorship.

The enumeration was made inside the defense of the governance/authorship distinction; it was not formalized as a specification with operational components, and the four roles were not articulated independently. Downstream work invoking CKS's authorship structure has to reconstruct the role distinctions from a list embedded in another note's argument, which invites either compression — *authorship* treated as a single undifferentiated activity — or fragmentation — *authorship* split into many ad-hoc categories beyond the four. The four architectural-element categories CKS deployments require (substrate structure, orchestration logic, cell behavior, initial state) are also not visibly tied to the role categories that produce them.

This note formalizes the four roles as a standalone specification. The roles correspond to the four element categories CKS deployments require to operate; each is operationally distinguishable, instantiates specific Series A architectural commitments, and typically requires role-specific expertise. The note pairs with the standalone governance-vs-authorship distinction (planned A2.65) and the architectural-commitments-producing-accessibility note (planned A2.67) to elaborate the parent note's commitments at full operational depth. (A terminological note: the parent note used "cell construction" for the third role; this note uses "cell authoring" for parallelism with schema, rule, and initial-content authoring. The substantive scope is identical.)

2. Schema authoring, defined precisely

Schema authoring produces substrate structure — the architectural definition of what substrate content looks like for a specific deployment. Schema authoring instantiates the substrate-layer commitments of the substrate-cell boundary (planned A2.08; source paper §2.1) for a particular deployment. It includes:

- (a) **Entity-type definition.** The kinds of entities the substrate carries — decisions, contradictions, plans, observations, rationale, and other coordination objects appropriate to the deployment.
- (b) **Relationship definition.** The kinds of relationships between entities — antecedent references, contradiction relationships, decision-rationale links, and other relations that the substrate's coordination semantics require.
- (c) **Field-constraint definition.** The constraints on substrate-content fields — validity rules, format requirements, allowed values, and any conflict-preservation semantics that apply at the field level.
- (d) **Schema-evolution rules.** How the schema changes over time as the deployment matures, including which evolutions preserve substrate addressability and which require structured migration.

The role typically requires expertise in domain modeling, data architecture, and the coordination patterns the deployment supports. The expertise is rarely held by non-specialists; entity-relationship modeling, schema-evolution patterns, and coordination-requirements analysis are specialist competencies. The architectural commitment to non-specialist governance (Li, 28 April 2026) does not extend to schema authoring; non-specialists govern *over* substrate content within schemas authored by specialists, and the asymmetry is what the parent note's section 3 names.

3. Rule authoring, defined precisely

Rule authoring produces orchestration rules — the rules that govern cell behavior over substrate. Rule authoring is the only one of the four roles with a *hybrid* character: the authoring act occurs within the human-governed authority context as a Moment 1 governance exercise (Li, 24 April 2026 §4; planned A2.04), and the action also produces new rule content that itself becomes substrate content with substrate authority (planned A2.46). Rule authoring is therefore both governance (in its authority context) and authorship (in its content production), and the four-role specification has to recognize both dimensions.

Rule authoring includes:

- (a) **Rule content.** The substantive logic of rules — what cells can read, what cells can write, what cells produce, under what conditions, and how preserved conflicts are handled when the rule encounters them.
- (b) **Rule applicability.** The substrate scopes within which each rule applies, and the boundary conditions under which the rule's effect ends.
- (c) **Rule provenance.** The authoring metadata that accompanies rules as substrate content, providing the path-retraceability properties the source paper imports from Rajabi and Kafaie (§3.1).
- (d) **Rule version history.** How rules change as the deployment evolves, with prior rule versions preserved or retired under the substrate's governance.

The authorship side of rule authoring typically requires expertise in coordination logic, rule-expression formats, cell-behavior specification, and the deployment's cell architecture. The non-specialist-governance commitment per the parent note applies to the governance dimension of rule authoring (a non-specialist may exercise the authority to commit a rule); the authorship dimension typically requires specialist expertise to produce rule content well-formed enough to function operationally.

4. Cell authoring, defined precisely

Cell authoring produces cell behavior under rules — the specifications for how cells operate over substrate. Cell authoring instantiates the cell-layer commitments of the substrate-cell boundary (planned A2.09; source paper §4.1) for specific cells in the deployment. It includes:

- (a) **Cell scope definition.** What substrate scopes each cell operates over, consistent with the cell-layer's bounded-scope commitment.
- (b) **Cell operation specification.** What operations the cell performs — read patterns, write patterns, conflict-handling patterns — under the cell-layer's rule-governed-behavior commitment.
- (c) **Cell rule-binding.** Which rules the cell operates under, consistent with the AI-as-substrate-mediator commitment (source paper §4.2) that LLM-mediated cells write only under human-authored orchestration rules.
- (d) **Cell statelessness specification.** How the cell handles state across executions, consistent with the cell-layer's statelessness commitment that the substrate, not the cell, holds coordination state across executions.

The role typically requires expertise in cell-execution patterns, orchestration mechanics, the substrate-cell boundary, and the specific operations the deployment requires. The expertise is rarely held by non-specialists; cell construction is the most architecturally specialist of the four roles, sitting at the intersection of LLM mediation, orchestration logic, and host-environment integration. The non-specialist-governance commitment

is preserved without requiring non-specialist cell authoring: cells produce substrate content under governance, and governance over that substrate content is exercisable by non-specialists regardless of who authored the cells.

5. Initial content authoring, defined precisely

Initial content authoring produces the substrate content the deployment starts with. It includes:

- (a) **Initial entities.** The entities the deployment begins with, conformant to the schema authored by the role of section 2.
- (b) **Initial relationships.** The relationships between initial entities that the substrate's coordination semantics require.
- (c) **Initial decisions and rationale.** Decisions that pre-date the deployment's operational start, with the rationale and accountability vocabulary the substrate's path-retraceability commitment requires.
- (d) **Initial state.** The state of any coordination objects at deployment start that subsequent cell operation will operate over.

The role typically requires domain expertise in what the deployment is coordinating about, since initial content represents the substantive starting state. Domain expertise may be held by specialists in the deployment's subject domain — who may themselves be non-specialists in CKS architecture. The role is the authorship category most accessible to non-specialists in the architectural sense: drafting an initial entity is an exercise of the modify right against a substrate that already has its schema, rules, and cells in place, and the parent note's section 3 explicitly notes that this category sits closer to governance than the others. The architectural commitment is that governance is accessible regardless of who authored initial content, not that initial-content authoring is itself a governance act.

6. The four operational components of the specification

The four roles together constitute an architectural specification. The specification has four components, each holding uniformly across the four roles.

- (a) **Role-specific architectural element.** Each role produces a specific architectural element — schema (substrate structure), rules (orchestration logic), cells (cell behavior), initial content (initial substrate state). The four elements are architecturally distinct; producing one does not produce the others, and a deployment's substrate, rules, cells, and initial content each trace to one of the four roles having produced it.
- (b) **Role-specific expertise requirement.** Each role typically requires expertise specific to that role — domain modeling for schema, coordination logic for rules, cell-execution patterns for cells, domain expertise for initial content. Expertise in one role does not imply expertise in the others; the four roles do not collapse into a single "authorship" expertise category.
- (c) **Role-specific architectural-commitment-location.** Each role instantiates specific Series A commitments — substrate-layer commitments (planned A2.08) for schema, rule-as-governance-moment and rules-as-substrate-content (planned A2.04, A2.46) for rules, cell-layer commitments (planned A2.09) for cells, schema-conformant content for initial content. The commitment-locations are distinct; producing one element does not instantiate commitments belonging to the others.

(d) **Role-specific relationship to governance.** Schema authoring, cell authoring, and initial-content authoring are pure authorship — at the action level they have no governance dimension; their results become governable substrate content once produced, but the production itself is not a governance act. Rule authoring is hybrid — the authoring act is itself a Moment 1 governance exercise, and simultaneously produces new rule content that requires authorship expertise. The non-specialist-governance commitment applies uniformly to the governance dimension wherever it appears; the authorship side of each role typically requires role-specific expertise.

A specification satisfying all four components defines the four-authorship-role architecture. A specification that collapses any of (a)–(d) is not the four-role specification, even if it superficially names four roles.

7. What the four-role specification is not

The standalone specification can be overstated. Six bounds and four adjacent categorizations need to be named to keep it within its intended scope.

Six bounds on the specification. It does *not* claim that the four roles must be exercised by different humans — the same human may exercise multiple roles, and small deployments often see one human exercise all four. It does *not* claim equal expertise requirements across roles — schema and rule authoring typically require more architectural expertise than cell authoring under standard patterns, and initial-content authoring may require only domain expertise. It does *not* specify implementation patterns; deployments may use various tools and workflows for each role. It does *not* foreclose collaboration — multiple humans may collaborate on one role, e.g., authoring rules for different substrate scopes. It does *not* require all four roles to be exercised in every deployment — a deployment with empty initial state does not require initial-content authoring at start. It does *not* foreclose role-evolution over deployment lifetime — schema authoring typically concentrates at design time, rule authoring continues through operation, cell authoring at design and maintenance time, initial-content authoring at deployment start with subsequent content produced through cell operation under governance.

Four adjacent categorizations. The four authorship roles are *not* developer/operator (which distinguishes who builds the system from who runs it), *not* designer/implementer (which distinguishes planning from building), *not* architect/engineer (which distinguishes high-level design from detailed implementation), and *not* owner/user (which distinguishes authority over the system from interaction with it). The four authorship roles operate at the architectural-element-production level: each names what kind of architectural element is being produced, not who builds versus runs, plans versus implements, or has authority versus interacts. A single individual may be developer and operator while exercising different authorship roles at different times; conversely, a single authorship role may be split across multiple humans regardless of how those humans relate on the four adjacent axes. The owner/user distinction in particular operates at the governance level (per the human-governed commitment); the four authorship roles operate at the production level.

8. Failure modes that violate the specification

Eight failure modes name ways an implementation can fail by collapsing or conflating the four roles.

(a) **Treating schema and content as unified.** Substrate structure (schema) and substrate content are treated as one architectural element, fusing schema authoring and initial-content authoring; component (a) of section 6 fails.

(b) **Treating rules and cells as unified.** Orchestration rules and cell behavior are treated as one element, fusing rule authoring and cell authoring; rule-authoring expertise (coordination logic) and cell-authoring expertise (cell-execution patterns) are not distinguished, and the substrate-cell boundary is operationally weakened.

(c) **Embedded authorship through configuration.** Authorship is treated as configuration of a pre-built system rather than as production of architectural elements; the four roles are obscured because configuration appears unified at the deployment-feature level, and downstream workers cannot see which Series A commitment each configuration choice instantiates.

(d) **Fragmenting roles into ad-hoc categories.** Authorship is fragmented into many categories beyond the four — "data engineering," "ML engineering," "domain expertise," "operations," "platform engineering" — and the architectural commitment to four specific roles is replaced with operational fragmentation.

(e) **Collapsing rule-authoring's hybrid nature.** Rule authoring is treated as either pure governance (over-claiming non-specialist accessibility for the authoring of rule content) or pure authorship (denying that the authoring act occurs within the human-governed authority context); component (d) of section 6 fails.

(f) **Mode-dependent role-distinguishability.** Role-distinguishability is treated differently across the labor-allocation modes (planned A1.12) — e.g., schema authoring distinct when humans perform it directly but unified with content authoring when an LLM under rules performs it; the architectural commitment to mode-independent role-distinguishability is broken.

(g) **Implicit-authorship patterns.** Architectural elements are produced through implicit operational paths — schema emerging from initial content as generalization, rules emerging from cell behavior as inference — rather than through explicit authorship acts; the role categories are obscured and the path-retraceability commitment cannot operate over elements with no authoring provenance.

(h) **Cross-role expertise uniformization.** All authorship is treated as requiring uniform expertise — e.g., requiring all authors to hold full-stack expertise across schema, rules, cells, and content; component (b) of section 6 fails, and the four roles collapse at the expertise level even if they remain nominally distinct at the element level.

9. Operational test

A system instantiates the four-authorship-role specification if and only if all of the following are true.

1. **Schema authoring** is operationally distinguishable as producing substrate structure per section 2 — entity-type definition, relationship definition, field-constraint definition, schema-evolution rules.
2. **Rule authoring** is operationally distinguishable as producing orchestration rules per section 3 — rule content, rule applicability, rule provenance, rule version history — *and* is recognized as the hybrid role with both a governance dimension (the Moment 1 authority context) and an authorship dimension (the production of new rule content).
3. **Cell authoring** is operationally distinguishable as producing cell behavior per section 4 — cell scope definition, cell operation specification, cell rule-binding, cell statelessness specification.

4. **Initial-content authoring** is operationally distinguishable as producing initial substrate content per section 5 — initial entities, initial relationships, initial decisions and rationale, initial state.

5. The four roles each satisfy section 6's four operational components — role-specific architectural element, role-specific expertise requirement, role-specific architectural-commitment-location, role-specific relationship to governance.

6. The four roles together account for the deployment's architectural elements: every element in the deployment's substrate, rules, cells, and initial content traces to one of the four roles having produced it.

A system that fails any of (1)–(6) does not instantiate the four-role specification in the architectural sense, even if it has operational authorship interfaces or nominally separates roles. A system that satisfies all six instantiates the specification by construction.

10. Why naming the four roles as standalone is load-bearing

Implementations under pressure to simplify operational processes consistently drift toward role-collapse patterns. The drift is steady because operational simplicity is rhetorically appealing — fewer roles is easier to staff, fewer expertise requirements is easier to recruit — and commercially familiar — vendors typically reduce role complexity to broaden their market. Implementations that drift away from the four-role specification produce systems where authorship is operationally undifferentiated, with downstream consequences in expertise-allocation failures, architectural-commitment-location ambiguity, governance-vs-authorship distinction failures, and labor-allocation framework failures (the three labor modes cannot be applied across the authorship categories when the categories are not distinguished).

The four-role specification is also load-bearing for several CKS commitments beyond the parent note: the substrate-cell boundary depends on the two layers being instantiable through distinct authorship roles (schema for the substrate layer, cell for the cell layer); the rule-authoring-as-governance-moment commitment depends on rule authoring being recognizable as a distinct role with a distinct relationship to governance; the Category-4 source-of-truth commitment depends on rule authoring producing substrate-resident rule content with substrate authority; and the labor-allocation framework depends on the four roles being distinguishable so its three modes can be applied across them.

Naming the four roles as a standalone architectural specification — with each role formalized in sections 2–5, the four operational components in section 6, the limitations and adjacent categorizations in section 7, the eight failure modes in section 8, and the operational test in section 9 — gives downstream readers the precise specification of authorship the architecture requires to be distinguished. The note pairs with the standalone governance-vs-authorship distinction (planned A2.65) and with the three architectural commitments producing accessibility (planned A2.67) to elaborate the parent foundational note (Li, 28 April 2026) at full operational depth.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Companion notes

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Non-Specialist Governance, Not Non-Specialist Authorship: What the Coordination Knowledge Substrate Pattern Makes Available to Whom*. 28 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Four Authorship Roles — Schema, Rule, Cell, Initial Content: A Standalone Treatment of the Authorship Action Categories in the Coordination Knowledge Substrate Pattern*. 5 May 2026. ORCID: 0009-0004-8065-3235.